FOUNDATION OF JAVA LAB

Exception Handling — Java Code Solutions

Solutions for Questions 1–15 from the uploaded lab sheet

Each program is standalone. Save each code section in a separate .java file using the public class name shown in that program.

Note: Q9 intentionally triggers an ArithmeticException to demonstrate exception propagation.

Q1. Handling ArithmeticException

```
import java.util.Scanner;

public class Q1ArithmeticException {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Enter first integer: ");
            int a = Integer.parseInt(sc.nextLine());
            System.out.print("Enter second integer: ");
            int b = Integer.parseInt(sc.nextLine());
            System.out.println("Result = " + (a / b));
        } catch (ArithmeticException e) {
            System.out.println("Error: Cannot divide by zero.");
        } catch (NumberFormatException e) {
            System.out.println("Error: Please enter valid integers.");
        } finally {
            sc.close();
        }
    }
}
```

Q2. Handling ArrayIndexOutOfBoundsException

```
import java.util.Scanner;

public class Q2ArrayIndex {
    public static void main(String[] args) {
        int[] values = {10, 20, 30, 40, 50};
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Enter an index (0-4): ");
            int index = Integer.parseInt(sc.nextLine());
            System.out.println("Element = " + values[index]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Error: Index must be between 0 and 4.");
        } catch (NumberFormatException e) {
            System.out.println("Error: Please enter a valid integer.");
        } finally {
            sc.close();
        }
    }
}
```

Q3. Handling NumberFormatException

```
public class Q3NumberFormat {
    public static void main(String[] args) {
        String text = "12x";
        try {
            int number = Integer.parseInt(text);
            System.out.println("Number = " + number);
        } catch (NumberFormatException e) {
            System.out.println("Error: The string is not a valid integer.");
        }
    }
}
```

Q4. Multiple catch Blocks

```
import java.util.Scanner;

public class Q4MultipleCatch {
    public static void main(String[] args) {
        int[] values = {10, 20, 30, 40, 50};
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Enter first number: ");
            int a = Integer.parseInt(sc.nextLine());
            System.out.print("Enter second number: ");
            int b = Integer.parseInt(sc.nextLine());
```

```
            System.out.println("Division = " + (a / b));

            System.out.print("Enter array index (0-4): ");
            int index = Integer.parseInt(sc.nextLine());
            System.out.println("Element = " + values[index]);
        } catch (ArithmeticException e) {
            System.out.println("Error: Cannot divide by zero.");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Error: Invalid array index.");
        } catch (NumberFormatException e) {
            System.out.println("Error: Please enter valid integers.");
        } finally {
            sc.close();
        }
    }
}
```

Q5. Using finally

```
import java.util.Scanner;

public class Q5Finally {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Enter numerator: ");
            int a = Integer.parseInt(sc.nextLine());
            System.out.print("Enter denominator: ");
            int b = Integer.parseInt(sc.nextLine());
            System.out.println("Result = " + (a / b));
        } catch (ArithmeticException e) {
            System.out.println("Error: Cannot divide by zero.");
        } catch (NumberFormatException e) {
            System.out.println("Error: Please enter valid integers.");
        } finally {
            System.out.println("Program execution completed.");
            sc.close();
        }
    }
}
```

Q6. Using throw

```
import java.util.Scanner;

public class Q6Throw {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Enter student's marks (0-100): ");
            int marks = Integer.parseInt(sc.nextLine());
            if (marks < 0 || marks > 100) {
                throw new IllegalArgumentException("Marks must be from 0 to 100.");
            }
            System.out.println("Valid marks: " + marks);
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        } finally {
            sc.close();
        }
    }
}
```

Q7. Using throws

```
import java.util.Scanner;

public class Q7Throws {
    static void checkAge(int age) throws Exception {
        if (age < 18) {
            throw new Exception("Age must be at least 18.");
        }
        System.out.println("Age is valid.");
```

```
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Enter age: ");
            int age = Integer.parseInt(sc.nextLine());
            checkAge(age);
        } catch (NumberFormatException e) {
            System.out.println("Error: Please enter a valid age.");
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        } finally {
            sc.close();
        }
    }
}
```

Q8. Nested try-catch

```
import java.util.Scanner;

public class Q8NestedTryCatch {
    public static void main(String[] args) {
        int[] values = {5, 10, 15, 20, 25};
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Enter numerator: ");
            int a = Integer.parseInt(sc.nextLine());
            System.out.print("Enter denominator: ");
            int b = Integer.parseInt(sc.nextLine());
            System.out.println("Division = " + (a / b));

            try {
                System.out.print("Enter array index (0-4): ");
                int index = Integer.parseInt(sc.nextLine());
                System.out.println("Element = " + values[index]);
            } catch (ArrayIndexOutOfBoundsException e) {
                System.out.println("Inner error: Invalid array index.");
            } catch (NumberFormatException e) {
                System.out.println("Inner error: Index must be an integer.");
            }
        } catch (ArithmeticException e) {
            System.out.println("Outer error: Cannot divide by zero.");
        } catch (NumberFormatException e) {
            System.out.println("Outer error: Please enter valid integers.");
        } finally {
            sc.close();
        }
    }
}
```

Q9. Exception Propagation

```
public class Q9ExceptionPropagation {
    static void method3() {
        int result = 10 / 0; // Generates ArithmeticException
        System.out.println(result);
    }

    static void method2() {
        method3();
    }

    static void method1() {
        method2();
    }

    public static void main(String[] args) {
        try {
            method1();
        } catch (ArithmeticException e) {
            System.out.println("Exception handled in main: " + e);
        }
    }
}
```

Q10. User-Defined Exception

```
import java.util.Scanner;

class InvalidMarksException extends Exception {
    InvalidMarksException(String message) {
        super(message);
    }
}

public class Q10UserDefinedException {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Enter marks (0-100): ");
            int marks = Integer.parseInt(sc.nextLine());
            if (marks < 0 || marks > 100) {
                throw new InvalidMarksException("Marks must be from 0 to 100.");
            }
            System.out.println("Valid marks: " + marks);
        } catch (InvalidMarksException e) {
            System.out.println("Error: " + e.getMessage());
        } catch (NumberFormatException e) {
            System.out.println("Error: Enter marks as a whole number.");
        } finally {
            sc.close();
        }
    }
}
```

Q11. Bank Withdrawal System

```
import java.util.Scanner;

class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String message) {
        super(message);
    }
}

public class Q11BankWithdrawal {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Enter account balance: ");
            double balance = Double.parseDouble(sc.nextLine());
            System.out.print("Enter withdrawal amount: ");
            double amount = Double.parseDouble(sc.nextLine());

            if (balance < 0) {
                throw new IllegalArgumentException("Balance cannot be negative.");
            }
            if (amount < 0) {
                throw new IllegalArgumentException("Withdrawal cannot be negative.");
            }
            if (amount > balance) {
                throw new InsufficientBalanceException("Insufficient account balance.");
            }

            balance -= amount;
            System.out.println("Withdrawal successful.");
            System.out.println("Remaining balance = " + balance);
        } catch (InsufficientBalanceException e) {
            System.out.println("Error: " + e.getMessage());
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        } catch (NumberFormatException e) {
            System.out.println("Error: Enter valid numeric values.");
        } finally {
            System.out.println("Bank transaction completed.");
            sc.close();
        }
    }
}
```

Q12. Hospital Patient Age Validation

```
import java.util.Scanner;

class InvalidPatientAgeException extends Exception {
    InvalidPatientAgeException(String message) {
        super(message);
    }
}

public class Q12PatientAge {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Enter patient name: ");
            String name = sc.nextLine();
            System.out.print("Enter patient age: ");
            int age = Integer.parseInt(sc.nextLine());

            if (age < 0 || age > 120) {
                throw new InvalidPatientAgeException("Age must be between 0 and 120.");
            }
            System.out.println("Registration successful for " + name + ".");
        } catch (InvalidPatientAgeException e) {
            System.out.println("Error: " + e.getMessage());
        } catch (NumberFormatException e) {
            System.out.println("Error: Age must be a whole number.");
        } finally {
            sc.close();
        }
    }
}
```

Q13. Pharmaceutical Drug Dosage Validation

```
import java.util.Scanner;

class InvalidDosageException extends Exception {
    InvalidDosageException(String message) {
        super(message);
    }
}

public class Q13DosageValidation {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Enter patient name: ");
            String patient = sc.nextLine();
            System.out.print("Enter drug name: ");
            String drug = sc.nextLine();
            System.out.print("Enter dosage in mg (1-1000): ");
            int dosage = Integer.parseInt(sc.nextLine());

            if (dosage <= 0 || dosage > 1000) {
                throw new InvalidDosageException("Dosage must be between 1 and 1000 mg.");
            }
            System.out.println("Dosage validated for " + patient + " - " + drug + ".");
        } catch (InvalidDosageException e) {
            System.out.println("Error: " + e.getMessage());
        } catch (NumberFormatException e) {
            System.out.println("Error: Dosage must be a whole number.");
        } finally {
            System.out.println("Dosage validation completed.");
            sc.close();
        }
    }
}
```

Q14. Online Examination System

```
import java.util.Scanner;
```

```
class InvalidExamMarksException extends Exception {
    InvalidExamMarksException(String message) {
        super(message);
    }
}

public class Q14OnlineExam {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Enter student's marks (0-100): ");
            int marks = Integer.parseInt(sc.nextLine());

            if (marks < 0 || marks > 100) {
                throw new InvalidExamMarksException("Marks must be from 0 to 100.");
            }
            if (marks >= 40) {
                System.out.println("PASS");
            } else {
                System.out.println("FAIL");
            }
        } catch (InvalidExamMarksException e) {
            System.out.println("Error: " + e.getMessage());
        } catch (NumberFormatException e) {
            System.out.println("Error: Enter marks as a whole number.");
        } finally {
            System.out.println("Exam evaluation completed.");
            sc.close();
        }
    }
}
```

Q15. Pharmacy Inventory System

```
import java.util.Scanner;

class InvalidQuantityException extends Exception {
    InvalidQuantityException(String message) {
        super(message);
    }
}

class InsufficientMedicineStockException extends Exception {
    InsufficientMedicineStockException(String message) {
        super(message);
    }
}

public class Q15PharmacyInventory {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Enter medicine name: ");
            String medicine = sc.nextLine();
            System.out.print("Enter available quantity: ");
            int available = Integer.parseInt(sc.nextLine());
            System.out.print("Enter required quantity: ");
            int required = Integer.parseInt(sc.nextLine());

            if (available < 0 || required < 0) {
                throw new InvalidQuantityException("Quantities cannot be negative.");
            }
            if (required > available) {
                throw new InsufficientMedicineStockException(
                    "Required quantity is greater than available stock.");
            }

            System.out.println("Dispensing " + required + " units of " + medicine + ".");
            System.out.println("Remaining stock = " + (available - required));
        } catch (InvalidQuantityException e) {
            System.out.println("Error: " + e.getMessage());
        } catch (InsufficientMedicineStockException e) {
            System.out.println("Error: " + e.getMessage());
        } catch (NumberFormatException e) {
            System.out.println("Error: Quantities must be whole numbers.");
        } finally {
            System.out.println("Inventory transaction completed.");
            sc.close();
```

```
        }
    }
}
```